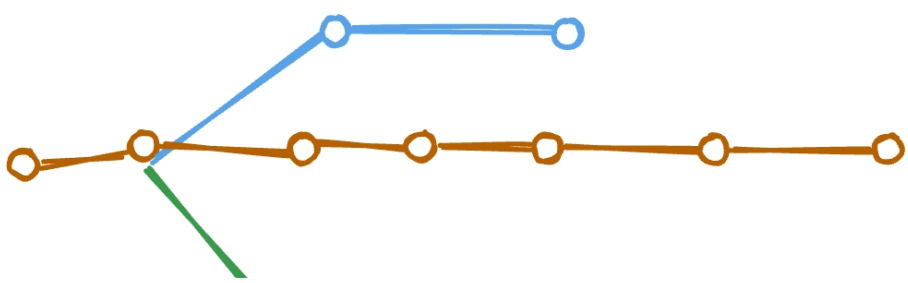
Branched in git

> **Learn and community**
>
> Check out our wide range of our courses.
> We have a Discord community where you can ask questions and get help from the community.

Branches in git

Branches are a way to work on different versions of a project at the same time. They allow you to create a separate line of development that can be worked on independently of the main branch. This can be useful when you want to make changes to a project without affecting the main branch or when you want to work on a new feature or bug fix.





Some developers can work on Header, some can work on Footer, some can work on Content, and some can work on Layout. This is a good example of how branches can be used in git.

HEAD in git

The HEAD is a pointer to the current branch that you are working on. It points to the latest commit in the current branch. When you create a new branch, it is automatically set as the HEAD of that branch.

> the default branch used to be master, but it is now called main. There is nothing special about main, it is just a convention.

Creating a new branch

To create a new branch, you can use the following command:

```
git branch
git branch bug-fix
git switch bug-fix
git log
git switch master
git switch -c dark-mode
git checkout orange-mode
```

Some points to note:

- `git branch` - This command lists all the branches in the current repository.
- `git branch bug-fix` - This command creates a new branch called `bug-fix`.
- `git switch bug-fix` - This command switches to the `bug-fix` branch.
- `git log` - This command shows the commit history for the current branch.
- `git switch master` - This command

- `git log` - This command shows the commit history for the current branch.
- `git switch master` - This command switches to the `master` branch.
- `git switch -c dark-mode` - This command creates a new branch called `dark-mode`. the `-c` flag is used to create a new branch.
- `git checkout orange-mode` - This command switches to the `orange-mode` branch.

> - Commit before switching to a branch
> - Go to .git folder and checkout to the HEAD file

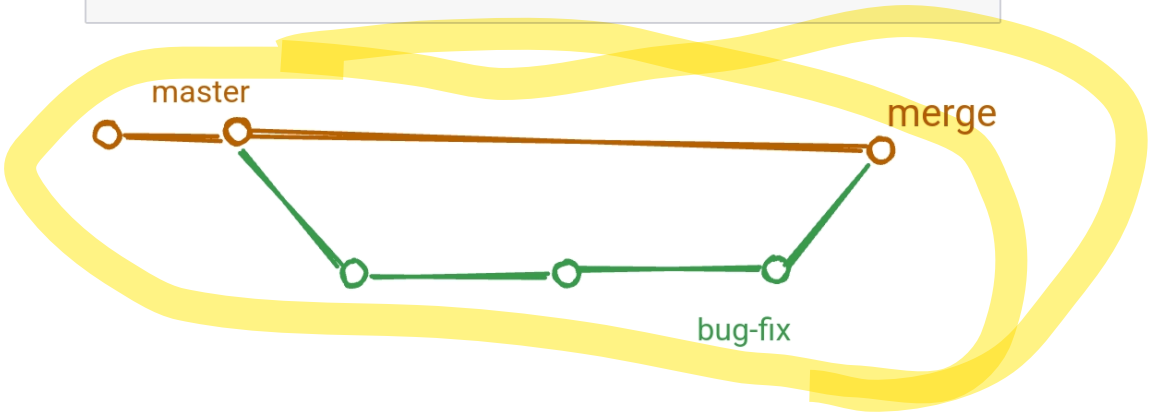
Merging branches

Fast-forward merge

This one is easy as branch that you are trying to merge is usually ahead and there are no conflicts.

When you are done working on a branch, you can merge it back into the main branch. This is done using the following command:

```
git checkout main
git merge bug-fix
```



Some points to note:

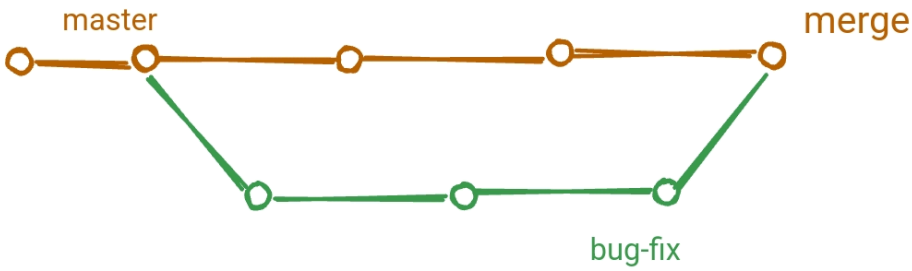
- `git checkout main` - This command switches to the `main` branch.
- `git merge bug-fix` - This command merges the `bug-fix` branch into the `main` branch.

This is a fast-forward merge. It means that the commits in the `bug-fix` branch are directly merged into the `main` branch. This can be useful when you want to merge a branch that has already been pushed to the remote repository.

Not fast-forward merge



Not fast-forward merge



In this type of merge, the master branch also worked and have some commits that are not in the `bug-fix` branch. This is a not fast-forward merge.

When you are done working on a branch, you can merge it back into the main branch. This is done using the following command:

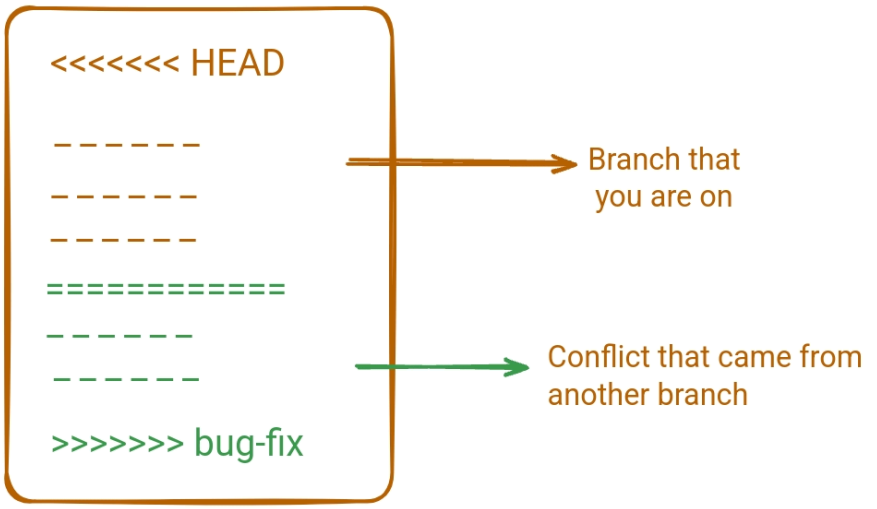
```
git checkout main
git merge bug-fix
```

after manual removal use these command :1)git add filename.txt 2) git commit -m "merging"

If the command are same, what is the difference between fast-forward and not fast-forward merge?

The difference is resolving the conflicts. In a fast-forward merge, there are no conflicts. But in a not fast-forward merge, there are conflicts, and there are no shortcuts to resolve them. You have to

The difference is resolving the conflicts. In a fast-forward merge, there are no conflicts. But in a not fast-forward merge, there are conflicts, and there are no shortcuts to resolve them. You have to manually resolve the conflicts. Decide, what to keep and what to discard. VSCode has a built-in merge tool that can help you resolve the conflicts.



Managing conflicts

There is no magic button to resolve conflicts. You have to manually resolve the conflicts. Decide, what to keep and what to discard. VSCode has a built-in merge tool that can help you resolve the conflicts. I personally use VSCode merge tool. Github also has a merge tool that can help you resolve the conflicts but most of the time I handle

Managing conflicts

There is no magic button to resolve conflicts. You have to manually resolve the conflicts. Decide, what to keep and what to discard. VSCode has a built-in merge tool that can help you resolve the conflicts. I personally use VSCode merge tool. Github also has a merge tool that can help you resolve the conflicts but most of the time I handle them in VSCode and it gives me all the options to resolve the conflicts.

Overall it sounds scary to beginners but it is not, it's all about communication and understanding the code situation with your team members.

Rename a branch

You can rename a branch using the following command:

method1:

```
git branch -m <old-branch-name> <new-bran
```

method2 :git branch -M new_branch

Delete a branch

You can delete a branch using the following command:

```
git branch -d <branch-name>
```

Checkout a branch

You can checkout a branch using the following command:

```
git checkout <branch-name>
```

Checkout a branch means that you are going to work on that branch. You can checkout any branch you want.

List all branches

You can list all branches using the following

List all branches

You can list all branches using the following command:

```
git branch
```

List all branches means that you are going to see all the branches in your repository.

Conclusion

In this section, we have learned about the different types of merges and how to resolve conflicts. We have also learned about the importance of branching and merging in Git and Github. By the end of this section, you should have a good understanding of how to use Git and Github effectively.